LabPulse

A lab equipment and incident tracker

Arka Ghosh | Group A | Roll 10110

DevOps final capstone

github.com/Godatcode/labpulse-devops-capstone

The application

A shared view of equipment condition and repair work

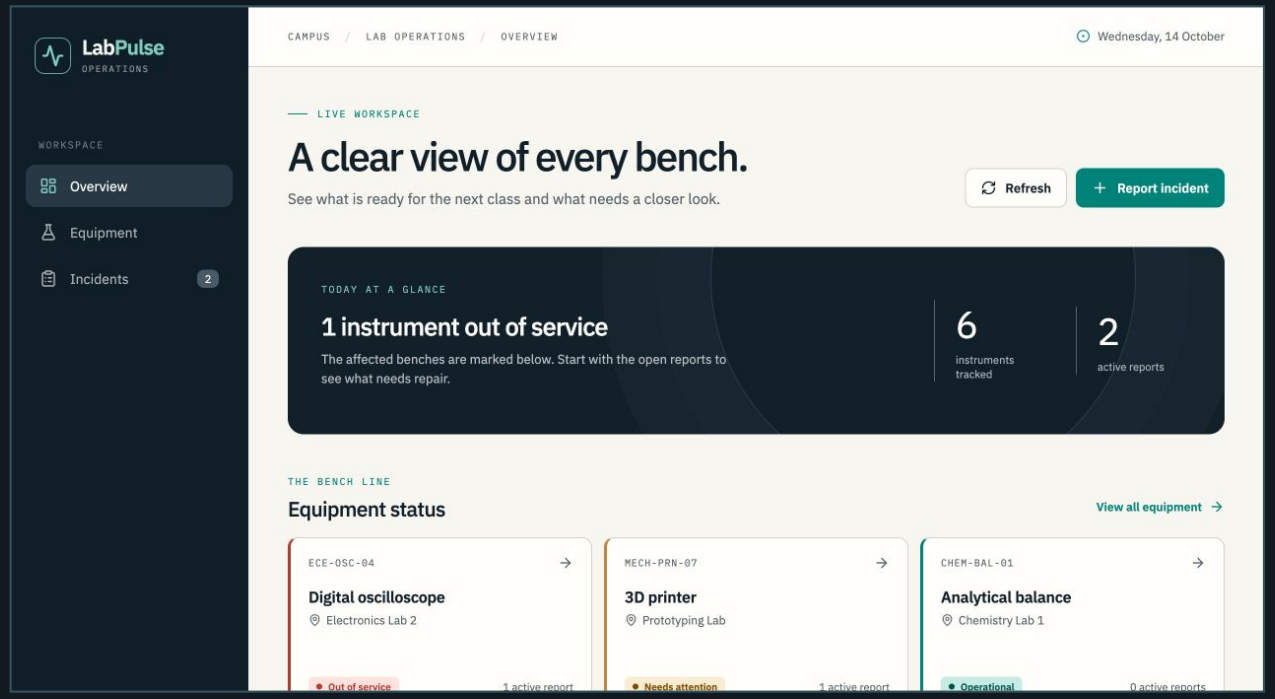
Technicians register equipment and report faults against it. Repair status moves from open to in progress to resolved.

Stack

React frontend

FastAPI backend

PostgreSQL + Alembic



Architecture

One app, two services, and one database

Browser > **Nginx frontend** > **FastAPI** > **PostgreSQL**

HTTP paths

/ serves React

/api routes to FastAPI

/health and /ready support checks

Metrics

Prometheus scrapes /metrics

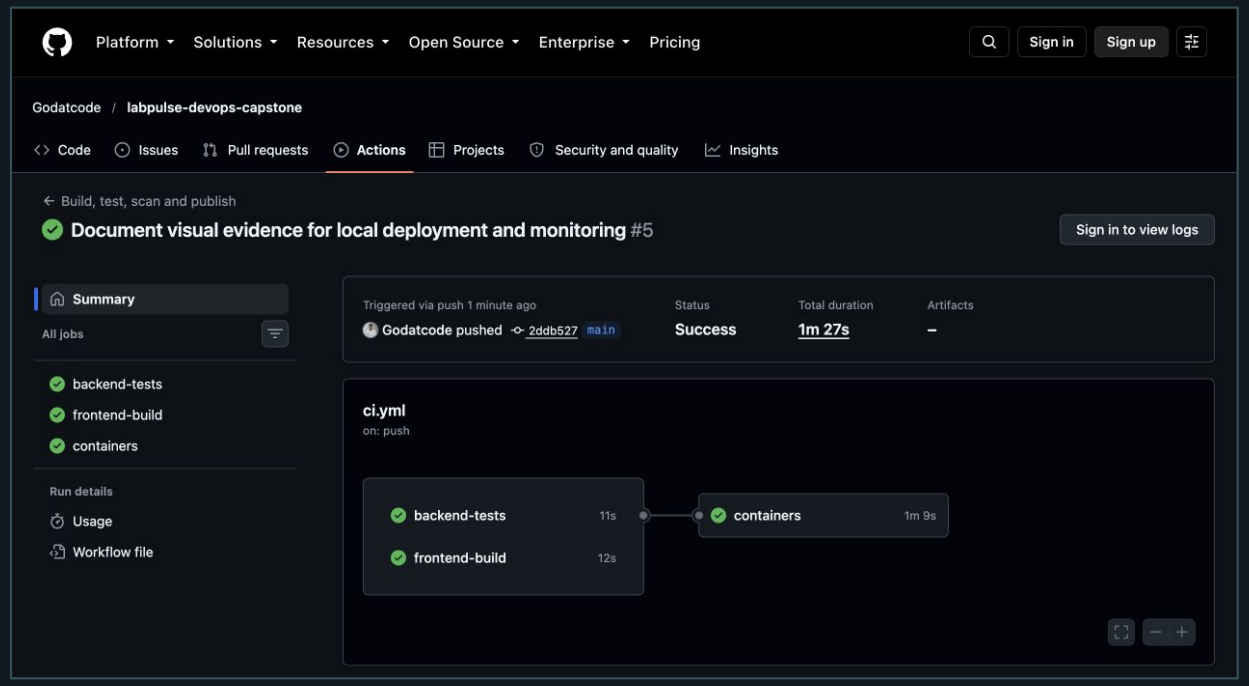
Grafana queries Prometheus

Requests and latency are visible

CI/CD pipeline

A main-branch push tests, scans, and publishes a traceable build

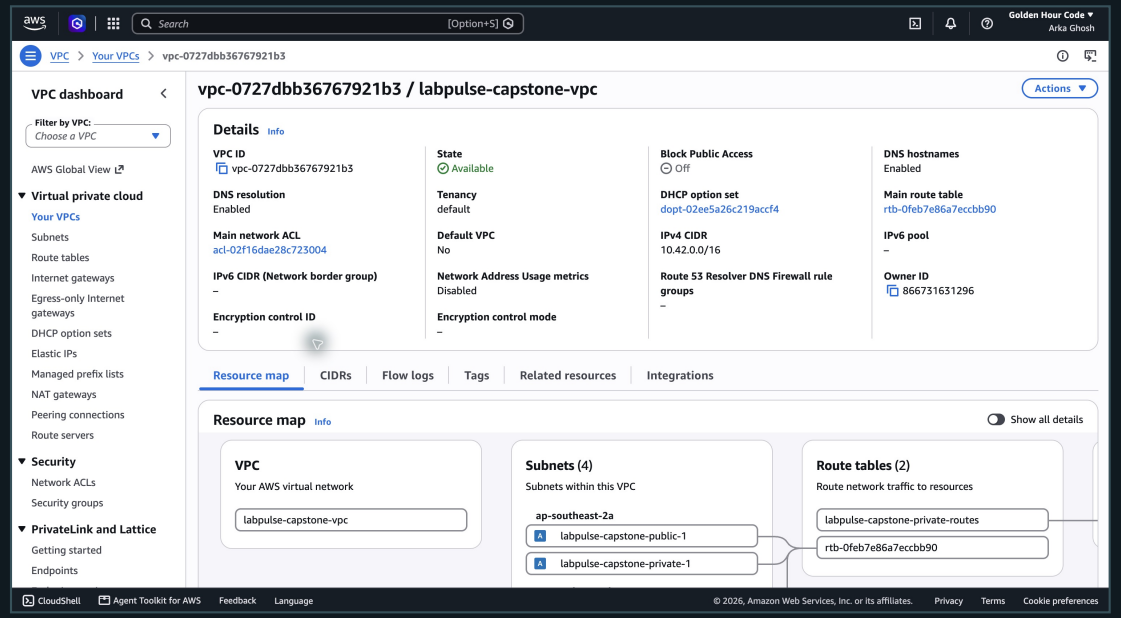
1. Pytest and Ruff check the API
2. Vite builds the frontend
3. Docker builds both images
4. Trivy scans both images
5. GHCR receives SHA-tagged images



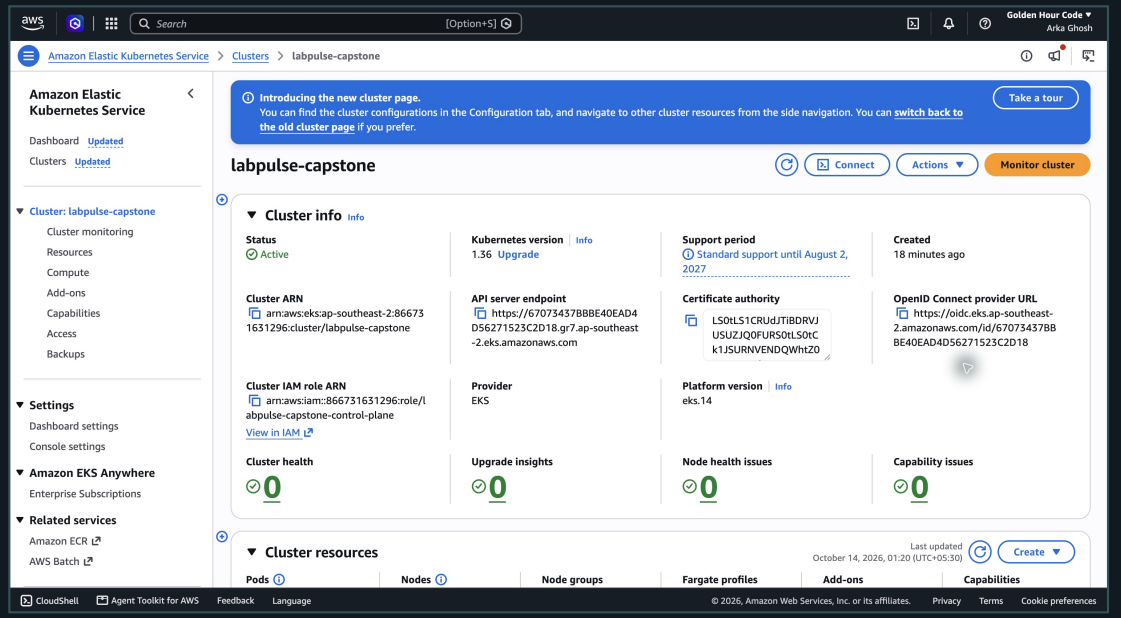
Successful run recorded in the repository evidence.

AWS infrastructure

Terraform provisioned the stack in ap-southeast-2



VPC: 2 public + 2 private subnets



EKS: cluster + managed node group

24 resources created; two t3.small workers reached Ready.

Terraform later destroyed all 24 resources to stop AWS costs.

Kubernetes and Helm

The local Kind run exercised the same application chart

Helm installed PostgreSQL,

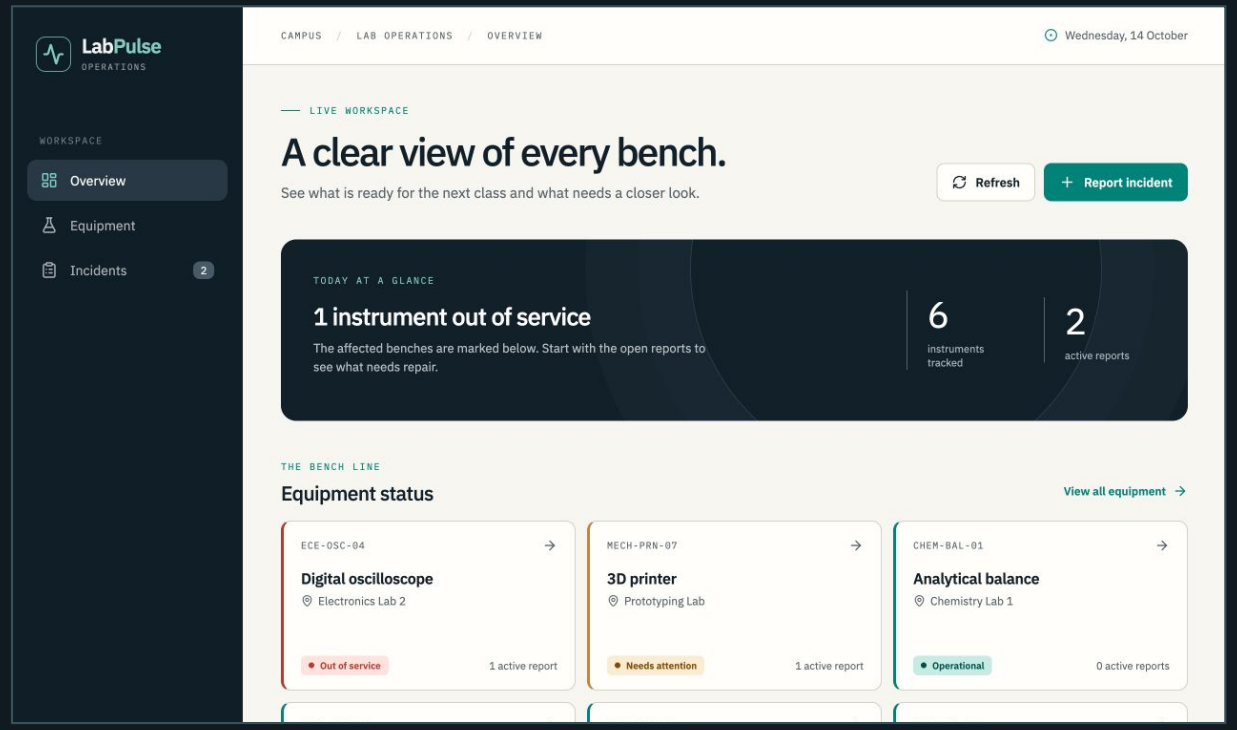
two API pods, and two web pods.

Services keep pod addressing stable.

Ingress routes / and /api.

All application pods were Running/Ready.

HPA reported CPU metrics for the backend.



GitOps with Argo CD

A Git commit reconciled the local Helm release

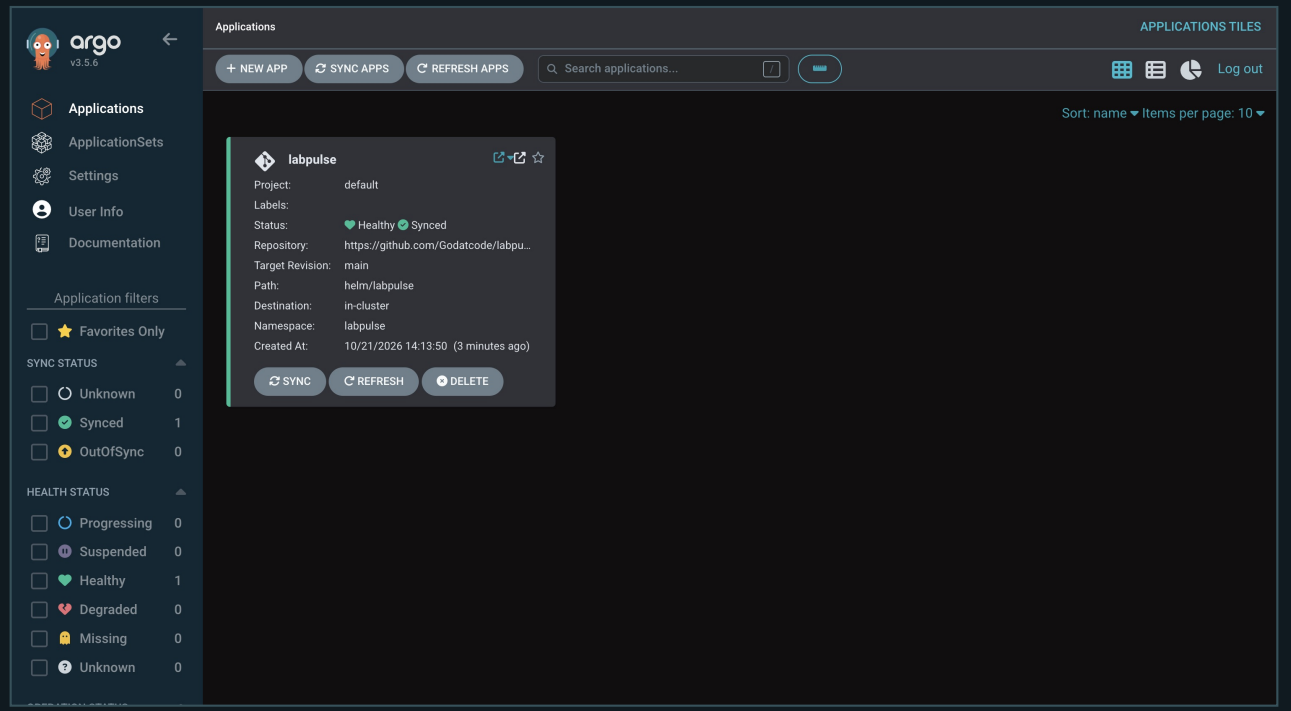
Application watches main.

Helm values live in Git.

Frontend replicas changed 2 -> 3.

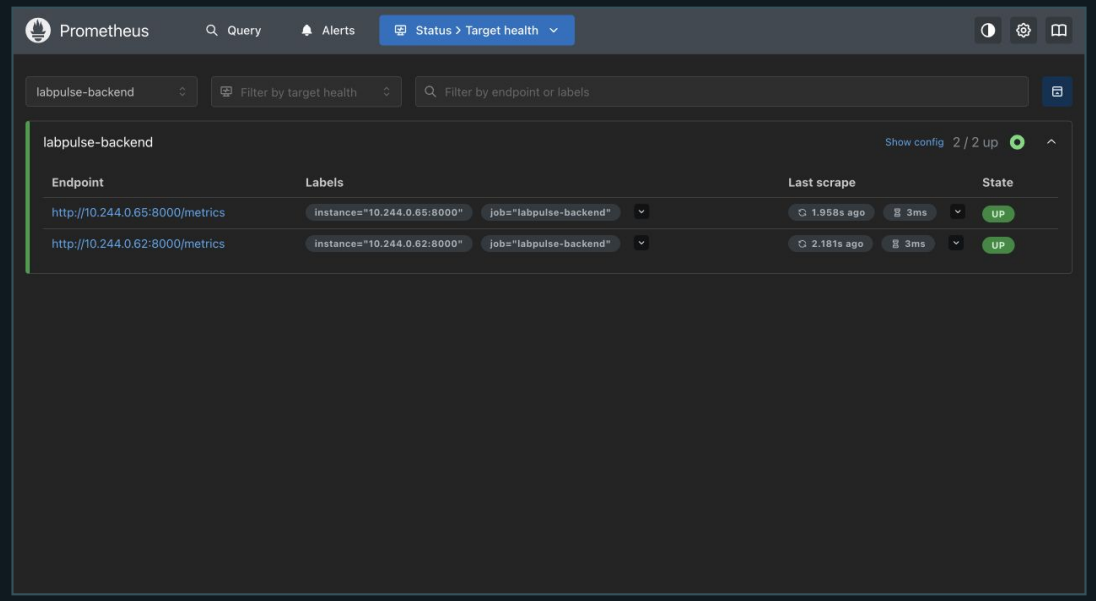
Argo CD reached Synced / Healthy.

The running Deployment reached 3/3 Ready.

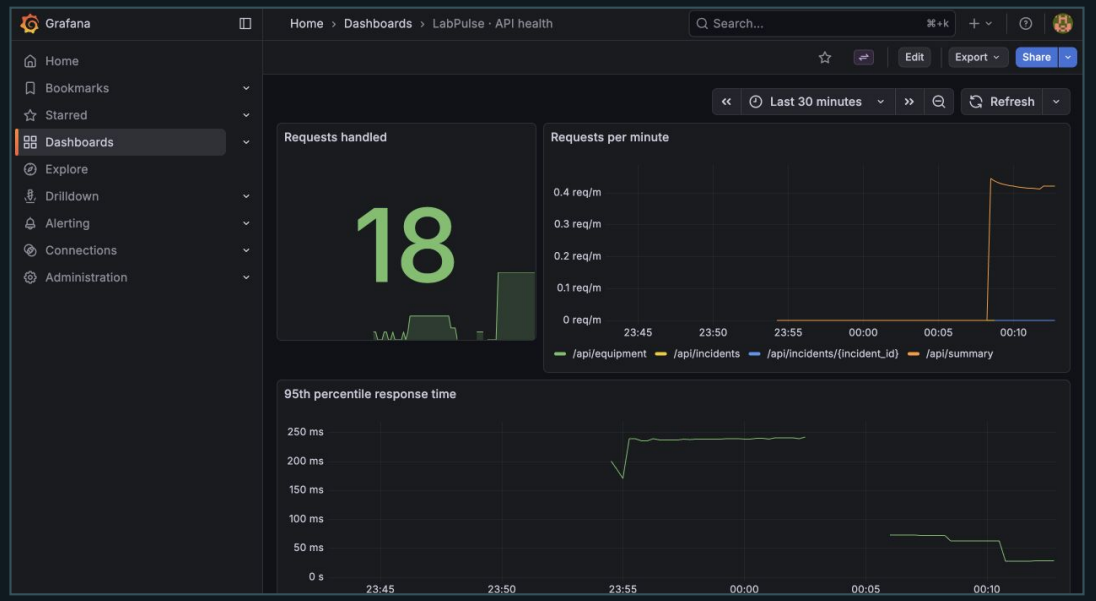


Observability

Application traffic appeared in Prometheus and Grafana



Prometheus: both backend targets UP



Grafana: request count, rate, p95 latency

FastAPI exposes /metrics; Prometheus scrapes both API pods every 15 seconds.

Tests and security

The pipeline checks behavior before it publishes images

10 API tests passed

Tests use an isolated SQLite database.
They cover equipment and incident flows
along with health checks.

Trivy blocked HIGH and CRITICAL findings

Both runtime images passed the final scan; new findings fail CI.

What I learned

The deployment evidence is reproducible, with clear limits

A green pipeline needs tests, image checks, and published tags.

Kubernetes health depends on readiness, Services, and routing.

A dashboard needs real traffic and a healthy scrape target.

Cloud evidence is useful only if cleanup is recorded too.

Current limit: no login or role checks; the AWS stack is destroyed.